Stride — Engineering Handoff

A single-page React prototype that tells a runner whether their training is working: a range-aware headline, one pace chart, "What changed?", a recommendation, recent runs, a training-pattern calendar and a run detail drawer.

1. How to run it

- **Prerequisites:** Node.js 18+ (required by Vite 5) and npm. No accounts, API keys or environment variables.
- **Run locally:** in the project folder, run `npm install`, then `npm run dev`, and open http://localhost:5173. Build: `npm run build`.
- **No-install option:** open `stride-prototype.html` in any browser. It's a self-contained build of the same source and works offline.
- **Dependencies:** `react` and `react-dom` 18.3. Dev only: `vite` 5 and `@vitejs/plugin-react`. No UI, chart or state libraries; charts are hand-built SVG.

2. Components, data model, state and shared styling

AREA	LOCATION	NOTES
Entry	`src/main.jsx`, `src/App.jsx`	Mounts `Dashboard`.
Homepage	`src/components/`: `PerformanceSummary`, `PerformanceChart`, `WhatChanged`, `NextStep`, `RecentRuns`	Rendered top to bottom in that order.
Detail views	`src/components/`: `TrainingPatternSheet` (+ `TrainingPatternChart`, `RunFilter`, `EmptyState`), `RunDetailDrawer`	Modals built on the shared `src/lib/useDialog.js` (focus trap, Escape, focus return, scroll lock).
Logic	`src/lib/metrics.js`	Pure functions: weekly series, best week, headline (`rangeStory`), insight, recommendation, recovery flags.
Data model	`src/data/runs.js`	`{ id, date: 'YYYY-MM-DD', type: 'fun' / 'tempo' / 'long', distance (mi), pace: 'm:ss' per mi, avgHr, rpe }`
Config and copy	`src/data/config.js`	Date ranges, run types, UI copy.
State	`src/components/Dashboard.jsx`	One owner for all state: `range`, `type`, `patternOpen`, `selectedRunId`. Passed down as props; no context or store.
Shared styling	`src/styles/global.css`	One stylesheet. Tokens at the top (`--bg`, `--text`, `--good` lime, `--warn` orange); one 760px breakpoint; `prefers-reduced-motion` respected.

3. What works, what's mocked, data assumptions and limitations

- **What works:** the date range (8, 4 or 2 weeks) recomputes every section; chart hover; training-pattern calendar with a run-type filter; run detail drawer; the empty state (Tempo + Last 2 weeks) with a working "Show last 4 weeks" recovery; full keyboard support. A 41-check browser test covered ranges, dialogs, focus and the empty state.
- **What's mocked:** all data is 35 fictional runs over 8 weeks (Aug 24 – Oct 18, 2026), tuned to tell one story: mileage 14 → 25 mi, best pace in week 4, slowing after. There is no backend, auth, persistence or network, so there are no loading, error or validation states.
- **Data assumptions:** weeks start Monday; weekly pace is distance-weighted; tempo and long runs count as hard; "slowing" means more than 5 sec/mi off best. A recovery opportunity is flagged only for a habit the runner kept from week 1 through their best week.
- **Most important limitations:**
  - The recovery rules are simple day-gap checks tuned to this dataset, not validated coaching logic.
  - Chart tooltips are mouse-only, the background isn't `inert` behind dialogs, and Safari needs Option+Tab to reach buttons.
  - Desktop-first: it reflows on phones but has no dedicated mobile design. US units and en-US dates only.
  - No unit or component tests in the repo.

4. Typical changes and next steps before production

Where to make a typical change:

- **Copy or labels:** `src/data/config.js` (`COPY`, `RUN_TYPES`, `DATE_RANGES`).
- **Headline or recommendation rules:** `rangeStory` and `recommendation` in `src/lib/metrics.js`. Components only render what these return.
- **New date range:** add an entry to `DATE_RANGES`; every section picks it up.
- **Colors or spacing:** the tokens at the top of `src/styles/global.css`.
- **Real data:** replace `src/data/runs.js` with an API call that returns the same shape; `metrics.js` stays unchanged.

What I'd address next, before production:

1. Replace the mock data with a real source (API or wearable sync), adding loading, error and empty-account states.
2. Add unit tests for `metrics.js` and component tests for the dialogs and empty state, and run the browser checks in CI.
3. Validate the recovery and recommendation rules with a coach, using varied real training data.
4. Close the accessibility gaps (keyboard chart access, `inert` background) and run a screen-reader audit.
5. Add TypeScript types for the run model, and self-host the font.